ePGD DL & GenAI: Assignment (Apr 2026)

Sem II - Ashish Trivedi

Part A : CNN (Image classification). Compare efficiency of convolutional neural network (CNN) architectures trained from scratch and using transfer learning techniques.

> **Dataset Reference**: CIFAR-100 Dataset. which consists of 100 object classes. Each input sample is a 32×32 RGB image, and the model output is a categorical label corresponding to one of the 100 object categories. The dataset contains 50,000 training images and 10,000 test images.

1. **From Scratch CNN**: Clearly describe the architecture, including the number of convolutional layers, kernel sizes, activation functions, and pooling operations. Train the model and report the training and validation accuracy. Comment on the convergence behavior and overall performance.
2. **A2**: xyz
3. **A3**:
4. **A4**: xyz

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader


```

```
PyTorch libraries imported. Further steps would involve rewriting data loading, model definitions, and training loops using PyTorc
```

```
#A1-1
import torchvision
```

```
3
4 # Define transformations for CIFAR-100
5 # For training, include data augmentation (random crop and horizontal flip)
6 # For testing, just normalize
7 transform_train = transforms.Compose([
8     transforms.RandomCrop(32, padding=4),
9     transforms.RandomHorizontalFlip(),
10    transforms.ToTensor(),
11    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize to [-1, 1]
12 ])
13
14 transform_test = transforms.Compose([
15    transforms.ToTensor(),
16    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize to [-1, 1]
17 ])
18
19 # Load CIFAR-100 datasets
20 trainset = datasets.CIFAR100(root='./data', train=True, download=True, transform=transform_train)
21 testset = datasets.CIFAR100(root='./data', train=False, download=True, transform=transform_test)
22
23 # Define DataLoaders
24 batch_size = 64
25 trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True, num_workers=2)
26 testloader = DataLoader(testset, batch_size=batch_size, shuffle=False, num_workers=2)
27
28 print("PyTorch CIFAR-100 DataLoaders created with transformations and batch size.")
29 print(f"Number of training batches: {len(trainloader)}")
```

```
PyTorch CIFAR-100 DataLoaders created with transformations and batch size.
Number of training batches: 782
Number of test batches: 157
```

```python
# A1-2
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=100):
        super(SimpleCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1), # 3 input channels (RGB), 64 output channels
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # Output: 64x16x16

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2), # Output: 128x8x8

            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2)  # Output: 256x4x4
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(256 * 4 * 4, 1024), # Flatten 256x4x4 to 4096 features
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(1024, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1) # Flatten all dimensions except batch
        x = self.classifier(x)
        return x

# Instantiate the model and print its summary
model = SimpleCNN(num_classes=100)
print(model)

# Move model to GPU if available
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model.to(device)
print(f"Model moved to: {device}")
```

```
SimpleCNN(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=4096, out_features=1024, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=1024, out_features=100, bias=True)
  )
)
Model moved to: cuda:0
```

Custom CNN Model Summary (PyTorch)

```
# A1-3
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

num_epochs = 25 # You can adjust this

# Initialize lists to store metrics for plotting
train_losses = []
test_losses = []
train_accuracies = []
test_accuracies = []

print("Starting training...")

```

```
for epoch in range(num_epochs):
    model.train() # Set model to training mode
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    train_accuracy = 100 * correct_train / total_train
    avg_train_loss = running_loss / len(trainloader)

    # Validation phase
    model.eval() # Set model to evaluation mode
    correct_test = 0
    total_test = 0
    test_loss = 0.0
    with torch.no_grad():
        for data in testloader:
            images, labels = data
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
```

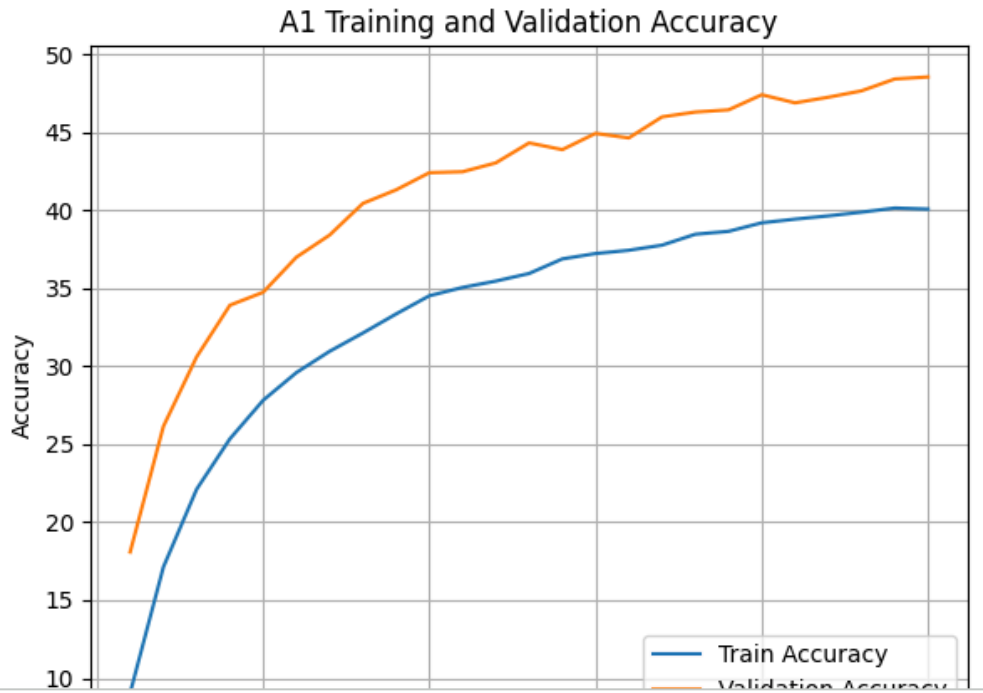
```
            total_test += labels.size(0)
            correct_test += (predicted == labels).sum().item()

    test_accuracy = 100 * correct_test / total_test
    avg_test_loss = test_loss / len(testloader)

    # Store metrics
    train_losses.append(avg_train_loss)
    test_losses.append(avg_test_loss)
    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)

    print(f'Epoch [{epoch+1}/{num_epochs}], ' \
          f'Train Loss: {avg_train_loss:.4f}, Train Acc: {train_accuracy:.2f}%, ' \
          f'Test Loss: {avg_test_loss:.4f}, Test Acc: {test_accuracy:.2f}%')

print('\nFinished Training PyTorch CNN Model')

# Evaluate the trained model on the test set one last time
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

final_accuracy = 100 * correct / total
```

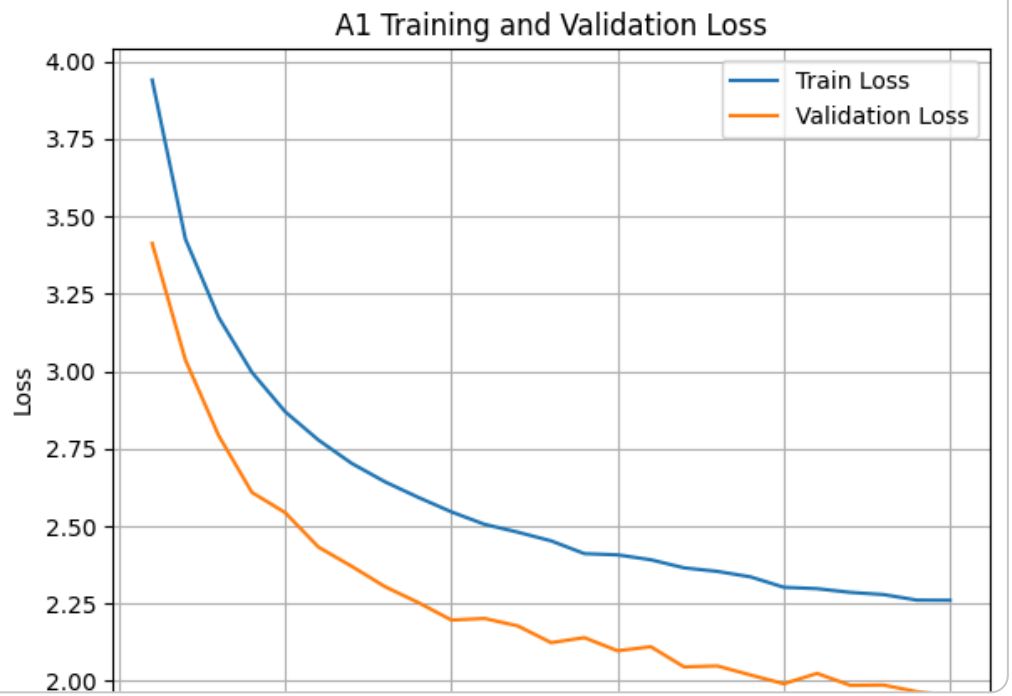
```
Starting training...
Epoch [1/25], Train Loss: 3.9405, Train Acc: 9.05%, Test Loss: 3.4130, Test Acc: 18.09%
Epoch [2/25], Train Loss: 3.4267, Train Acc: 17.13%, Test Loss: 3.0364, Test Acc: 26.15%
Epoch [3/25], Train Loss: 3.1741, Train Acc: 22.12%, Test Loss: 2.7924, Test Acc: 30.61%
Epoch [4/25], Train Loss: 2.9967, Train Acc: 25.35%, Test Loss: 2.6090, Test Acc: 33.92%
```

```
Epoch [5/25], Train Loss: 2.8695, Train Acc: 27.82%, Test Loss: 2.5438, Test Acc: 34.75%
Epoch [6/25], Train Loss: 2.7782, Train Acc: 29.60%, Test Loss: 2.4326, Test Acc: 37.00%
Epoch [7/25], Train Loss: 2.7029, Train Acc: 30.96%, Test Loss: 2.3706, Test Acc: 38.42%
Epoch [8/25], Train Loss: 2.6434, Train Acc: 32.14%, Test Loss: 2.3050, Test Acc: 40.45%
Epoch [9/25], Train Loss: 2.5932, Train Acc: 33.36%, Test Loss: 2.2537, Test Acc: 41.32%
Epoch [10/25], Train Loss: 2.5462, Train Acc: 34.53%, Test Loss: 2.1970, Test Acc: 42.42%
Epoch [11/25], Train Loss: 2.5060, Train Acc: 35.06%, Test Loss: 2.2021, Test Acc: 42.48%
Epoch [12/25], Train Loss: 2.4806, Train Acc: 35.47%, Test Loss: 2.1780, Test Acc: 43.05%
Epoch [13/25], Train Loss: 2.4526, Train Acc: 35.95%, Test Loss: 2.1241, Test Acc: 44.33%
Epoch [14/25], Train Loss: 2.4115, Train Acc: 36.89%, Test Loss: 2.1402, Test Acc: 43.90%
Epoch [15/25], Train Loss: 2.4075, Train Acc: 37.23%, Test Loss: 2.0978, Test Acc: 44.94%
Epoch [16/25], Train Loss: 2.3916, Train Acc: 37.44%, Test Loss: 2.1109, Test Acc: 44.65%
Epoch [17/25], Train Loss: 2.3655, Train Acc: 37.77%, Test Loss: 2.0457, Test Acc: 46.00%
Epoch [18/25], Train Loss: 2.3541, Train Acc: 38.47%, Test Loss: 2.0489, Test Acc: 46.31%
Epoch [19/25], Train Loss: 2.3364, Train Acc: 38.66%, Test Loss: 2.0196, Test Acc: 46.45%
Epoch [20/25], Train Loss: 2.3028, Train Acc: 39.21%, Test Loss: 1.9913, Test Acc: 47.42%
Epoch [21/25], Train Loss: 2.2983, Train Acc: 39.44%, Test Loss: 2.0248, Test Acc: 46.90%
Epoch [22/25], Train Loss: 2.2864, Train Acc: 39.65%, Test Loss: 1.9863, Test Acc: 47.26%
Epoch [23/25], Train Loss: 2.2793, Train Acc: 39.89%, Test Loss: 1.9872, Test Acc: 47.67%
Epoch [24/25], Train Loss: 2.2614, Train Acc: 40.15%, Test Loss: 1.9657, Test Acc: 48.43%
Epoch [25/25], Train Loss: 2.2609, Train Acc: 40.09%, Test Loss: 1.9521, Test Acc: 48.56%

Finished Training PyTorch CNN Model
Final Accuracy of the PyTorch CNN on the 10000 test images: 48.56%
```

```python
# A1-4
import matplotlib.pyplot as plt

epochs_range = range(1, num_epochs + 1)

# Plot training & validation accuracy values
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(epochs_range, train_accuracies, label='Train Accuracy')
plt.plot(epochs_range, test_accuracies, label='Validation Accuracy')
plt.title('A1 Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.grid(True)
```

```
17
18 # Plot training & validation loss values
19 plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
20 plt.plot(epochs_range, train_losses, label='Train Loss')
21 plt.plot(epochs_range, test_losses, label='Validation Loss')
22 plt.title('A1 Training and Validation Loss')
23 plt.ylabel('Loss')
24 plt.xlabel('Epoch')
25 plt.legend(loc='upper right')
26 plt.grid(True)
27
28 plt.tight_layout() # Adjust layout to prevent overlapping titles
29 plt.show()
```





A2. Robustness Evaluation: Additive Gaussian Noise

Now, we'll evaluate the trained `SimpleCNN` model's robustness by injecting additive Gaussian noise into a portion of the test images.

```python
# A2-1
import numpy as np

# Define noise parameters
noise_variance = 0.05  # sigma_square
noise_std_dev = np.sqrt(noise_variance) # sigma
noise_percentage = 0.50 # 50% of images will have noise

print(f"Evaluating robustness to noise with variance = {noise_variance} (std dev = {noise_std_dev:.4f}) on {noise_percentage

model.eval() # Set model to evaluation mode
correct_noisy = 0
total_noisy = 0

with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)

        # Randomly select images to add noise to
        batch_size = images.size(0)
        num_noisy_images = int(batch_size * noise_percentage)
        noisy_indices = np.random.choice(batch_size, num_noisy_images, replace=False)

        # Add Gaussian noise to selected images
        noise = torch.randn_like(images[noisy_indices]) * noise_std_dev
        images[noisy_indices] = images[noisy_indices] + noise

        # Clip values to ensure they remain within the normalized range [-1, 1]
        images[noisy_indices] = torch.clamp(images[noisy_indices], -1.0, 1.0)

        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total_noisy += labels.size(0)
        correct_noisy += (predicted == labels).sum().item()

noisy_accuracy = 100 * correct_noisy / total_noisy
print(f'Accuracy of the PyTorch CNN on the noisy test images: {noisy_accuracy:.2f}%')

```

```
Evaluating robustness to noise with variance = 0.05 (std dev = 0.2236) on 50.0% of test images...
Accuracy of the PyTorch CNN on the noisy test images: 29.40%
```

Noise Generation Process and Parameters:

- **Noise Type**: Additive Gaussian Noise
- **Mean**: 0
- **Variance ($sigma^2$)**: 0.05
- **Standard Deviation ($sigma$)**: $sqrt 0.05$

  $approx 0.2236$
- **Injection Rate**: 50% of the images in each test batch were randomly selected to have noise injected.
- **Normalization Range**: The noise was added to images already normalized to the range `[-1, 1]`.
- **Clipping**: After adding noise, the pixel values of the noisy images were clipped to `[-1, 1]` to maintain valid image representation.

The resulting accuracy on the noisy test set provides insight into the model's robustness under these specific noisy conditions.

- After adding noise in 50% of test data, test accuracy reduced from `48.56%` to `29.40%` . The model is "brittle". It has learned to recoginze the speicfic pixel patterns but hasn't learned to identify object based on shape or overall pixel distribution. So th enoise confused the model.

Double-click (or enter) to edit

A3 Improved training strategy to use noise images in training layer and mixed up images so that the test accuracy of noisy images remain high.

```
# A3 - 1
# Define the Gaussian Noise Transformation. In PyTorch code, we create a
# simple class to use inside torchvision.transforms.
```

```
class AddGaussianNoise(object):
    def __init__(self, mean=0.0, std=1.0, probability=0.5):
        self.std = std
        self.mean = mean
        self.probability = probability

    def __call__(self, tensor):
        # Only add noise to a percentage of the images (the "limited number" hint)
        if torch.rand(1).item() < self.probability:
            return tensor + torch.randn(tensor.size()) * self.std + self.mean
        return tensor
```

```
# A3 - 2
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    AddGaussianNoise(0.0, 0.05, probability=0.3), # 30% of images get noise
    transforms.Normalize((0.5071, 0.4867, 0.4408), (0.2675, 0.2565, 0.2761))
])

trainset = datasets.CIFAR100(root='./data', train=True, download=True, transform=transform_train)
trainloader = DataLoader(trainset, batch_size=128, shuffle=True, num_workers=2)
```

```
# A3 - 3
# Instantiate the model and print its summary
model2_A3 = SimpleCNN(num_classes=100)
print(model2_A3)

# Move model to GPU if available
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model2_A3.to(device)
print(f"Model moved to: {device}")
```

```
SimpleCNN(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=4096, out_features=1024, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=1024, out_features=100, bias=True)
  )
)
Model moved to: cuda:0
```

```
# A3 - 4
criterion_A3 = nn.CrossEntropyLoss()
optimizer_A3 = optim.Adam(model2_A3.parameters(), lr=0.001)

num_epochs_A3 = 25 # You can adjust this

# Initialize lists to store metrics for plotting for A3
train_losses_A3 = []
train_accuracies_A3 = []
clean_test_losses_A3 = []
clean_test_accuracies_A3 = []
noisy_test_losses_A3 = []
noisy_test_accuracies_A3 = []

print("Starting training for A3 model...")

for epoch in range(num_epochs_A3):
    model2_A3.train() # Set model to training mode
```

```
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer_A3.zero_grad()

        outputs = model2_A3(inputs)
        loss = criterion_A3(outputs, labels)
        loss.backward()
        optimizer_A3.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    train_accuracy = 100 * correct_train / total_train
    avg_train_loss = running_loss / len(trainloader)

    # Validation phase on CLEAN test set
    model2_A3.eval() # Set model to evaluation mode
    correct_clean_test = 0
    total_clean_test = 0
    clean_test_loss = 0.0
    with torch.no_grad():
        for data in testloader: # Using the standard testloader
            images, labels = data
            images, labels = images.to(device), labels.to(device)
            outputs = model2_A3(images)
            loss = criterion_A3(outputs, labels)
            clean_test_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total_clean_test += labels.size(0)
            correct_clean_test += (predicted == labels).sum().item()
```

```
58
59      clean_test_accuracy = 100 * correct_clean_test / total_clean_test
60      avg_clean_test_loss = clean_test_loss / len(testloader)
61
62      # Validation phase on NOISY test set (parameters same as A2 evaluation)
63      correct_noisy_test = 0
64      total_noisy_test = 0
65      noisy_test_loss = 0.0
66      noise_variance_eval = 0.05
67      noise_std_dev_eval = np.sqrt(noise_variance_eval)
68      noise_percentage_eval = 0.50
69
70      with torch.no_grad():
71          for data in testloader:
72              images_noisy_eval, labels_noisy_eval = data
73              images_noisy_eval, labels_noisy_eval = images_noisy_eval.to(device), labels_noisy_eval.to(device)
74
75              # Dynamically add noise for this evaluation pass
76              batch_size_eval = images_noisy_eval.size(0)
77              num_noisy_images_eval = int(batch_size_eval * noise_percentage_eval)
78              noisy_indices_eval = np.random.choice(batch_size_eval, num_noisy_images_eval, replace=False)
79
80              noise_eval = torch.randn_like(images_noisy_eval[noisy_indices_eval]) * noise_std_dev_eval
81              images_noisy_eval[noisy_indices_eval] = images_noisy_eval[noisy_indices_eval] + noise_eval
82              images_noisy_eval[noisy_indices_eval] = torch.clamp(images_noisy_eval[noisy_indices_eval], -1.0, 1.0)
83
84              outputs_noisy_eval = model2_A3(images_noisy_eval)
85              loss_noisy_eval = criterion_A3(outputs_noisy_eval, labels_noisy_eval)
86              noisy_test_loss += loss_noisy_eval.item()
87              _, predicted_noisy_eval = torch.max(outputs_noisy_eval.data, 1)
88              total_noisy_test += labels_noisy_eval.size(0)
89              correct_noisy_test += (predicted_noisy_eval == labels_noisy_eval).sum().item()
90
91      noisy_test_accuracy = 100 * correct_noisy_test / total_noisy_test
92      avg_noisy_test_loss = noisy_test_loss / len(testloader)
93
94      # Store metrics for A3
95      train_losses_A3.append(avg_train_loss)
96      train_accuracies_A3.append(train_accuracy)
```

```
    clean_test_losses_A3.append(avg_clean_test_loss)
    clean_test_accuracies_A3.append(clean_test_accuracy)
    noisy_test_losses_A3.append(avg_noisy_test_loss)
    noisy_test_accuracies_A3.append(noisy_test_accuracy)

    print(f'Epoch [{epoch+1}/{num_epochs_A3}], ' \
          f'Train Loss: {avg_train_loss:.4f}, Train Acc: {train_accuracy:.2f}%, ' \
          f'Clean Test Loss: {avg_clean_test_loss:.4f}, Clean Test Acc: {clean_test_accuracy:.2f}%, ' \
          f'Noisy Test Loss: {avg_noisy_test_loss:.4f}, Noisy Test Acc: {noisy_test_accuracy:.2f}%')

print('\nFinished Training A3 PyTorch CNN Model')

# Final Evaluation and Comparison for A3
print(f'Final Clean Test Accuracy for A3: {clean_test_accuracies_A3[-1]:.2f}%')
print(f'Final Noisy Test Accuracy for A3: {noisy_test_accuracies_A3[-1]:.2f}%')

import matplotlib.pyplot as plt

epochs_range_A3 = range(1, num_epochs_A3 + 1)

plt.figure(figsize=(15, 6))

# Plot A3 Training and Validation Accuracy
plt.subplot(1, 2, 1)
plt.plot(epochs_range_A3, train_accuracies_A3, label='Train Accuracy')
plt.plot(epochs_range_A3, clean_test_accuracies_A3, label='Clean Test Accuracy')
plt.plot(epochs_range_A3, noisy_test_accuracies_A3, label='Noisy Test Accuracy')
plt.title('A3 Training, Clean Test, and Noisy Test Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.grid(True)

# Plot A3 Training and Validation Loss
plt.subplot(1, 2, 2)
plt.plot(epochs_range_A3, train_losses_A3, label='Train Loss')
plt.plot(epochs_range_A3, clean_test_losses_A3, label='Clean Test Loss')
plt.plot(epochs_range_A3, noisy_test_losses_A3, label='Noisy Test Loss')
plt.title('A3 Training, Clean Test, and Noisy Test Loss')
```

```
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper right')
plt.grid(True)
```

```
Starting training for A3 model...
Epoch [1/25], Train Loss: 3.8151, Train Acc: 11.48%, Clean Test Loss: 3.7847, Clean Test Acc: 14.25%, Noisy Test Loss: 3.8079, Noi
Epoch [2/25], Train Loss: 3.1805, Train Acc: 22.26%, Clean Test Loss: 3.4358, Clean Test Acc: 18.69%, Noisy Test Loss: 3.4592, Noi
Epoch [3/25], Train Loss: 2.9039, Train Acc: 27.53%, Clean Test Loss: 3.3578, Clean Test Acc: 20.28%, Noisy Test Loss: 3.4321, Noi
Epoch [4/25], Train Loss: 2.7211, Train Acc: 31.08%, Clean Test Loss: 3.2697, Clean Test Acc: 21.33%, Noisy Test Loss: 3.3088, Noi
Epoch [5/25], Train Loss: 2.5665, Train Acc: 34.01%, Clean Test Loss: 3.1300, Clean Test Acc: 24.12%, Noisy Test Loss: 3.2640, Noi
Epoch [6/25], Train Loss: 2.4659, Train Acc: 36.26%, Clean Test Loss: 3.0512, Clean Test Acc: 25.95%, Noisy Test Loss: 3.1493, Noi
Epoch [7/25], Train Loss: 2.3689, Train Acc: 38.21%, Clean Test Loss: 3.0032, Clean Test Acc: 26.42%, Noisy Test Loss: 3.1431, Noi
Epoch [8/25], Train Loss: 2.2992, Train Acc: 39.56%, Clean Test Loss: 2.8498, Clean Test Acc: 29.60%, Noisy Test Loss: 2.9910, Noi
Epoch [9/25], Train Loss: 2.2286, Train Acc: 41.20%, Clean Test Loss: 2.9225, Clean Test Acc: 28.12%, Noisy Test Loss: 3.1102, Noi
Epoch [10/25], Train Loss: 2.1705, Train Acc: 42.28%, Clean Test Loss: 2.9092, Clean Test Acc: 28.77%, Noisy Test Loss: 3.0447, No
Epoch [11/25], Train Loss: 2.1308, Train Acc: 43.01%, Clean Test Loss: 2.7003, Clean Test Acc: 32.42%, Noisy Test Loss: 2.7919, No
Epoch [12/25], Train Loss: 2.1056, Train Acc: 43.61%, Clean Test Loss: 2.7150, Clean Test Acc: 32.01%, Noisy Test Loss: 2.8339, No
Epoch [13/25], Train Loss: 2.0422, Train Acc: 45.01%, Clean Test Loss: 2.6809, Clean Test Acc: 30.02%, Noisy Test Loss: 2.9188, No
Epoch [14/25], Train Loss: 2.0195, Train Acc: 45.50%, Clean Test Loss: 2.6124, Clean Test Acc: 33.94%, Noisy Test Loss: 2.7472, No
Epoch [15/25], Train Loss: 1.9867, Train Acc: 46.16%, Clean Test Loss: 2.7428, Clean Test Acc: 31.75%, Noisy Test Loss: 2.8204, No
Epoch [16/25], Train Loss: 1.9525, Train Acc: 46.89%, Clean Test Loss: 2.5993, Clean Test Acc: 34.84%, Noisy Test Loss: 2.7062, No
Epoch [17/25], Train Loss: 1.9252, Train Acc: 47.40%, Clean Test Loss: 2.6351, Clean Test Acc: 34.02%, Noisy Test Loss: 2.7689, No
Epoch [18/25], Train Loss: 1.9119, Train Acc: 48.24%, Clean Test Loss: 2.7519, Clean Test Acc: 32.08%, Noisy Test Loss: 2.9106, No
Epoch [19/25], Train Loss: 1.8840, Train Acc: 48.56%, Clean Test Loss: 2.6194, Clean Test Acc: 34.65%, Noisy Test Loss: 2.7577, No
Epoch [20/25], Train Loss: 1.8684, Train Acc: 48.68%, Clean Test Loss: 2.6669, Clean Test Acc: 33.41%, Noisy Test Loss: 2.7945, No
Epoch [21/25], Train Loss: 1.8434, Train Acc: 49.52%, Clean Test Loss: 2.6040, Clean Test Acc: 34.34%, Noisy Test Loss: 2.7376, No
Epoch [22/25], Train Loss: 1.8235, Train Acc: 49.90%, Clean Test Loss: 2.6293, Clean Test Acc: 33.84%, Noisy Test Loss: 2.7091, No
```

A4 Using Pre-trained VGG model

A4: CIFAR-100 Classification with Pre-trained VGG16 (Fixed Feature Extractor)

This section implements CIFAR-100 classification using a pre-trained VGG16 model. The convolutional layers of VGG16 will be used as fixed feature extractors, and a new classifier head will be trained.

A4 - Data Preparation for VGG16

```python
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# Define transformations for VGG16 (expects 224x224 input and ImageNet normalization)
transform_train_a4 = transforms.Compose([
    transforms.Resize(224), # Resize images to 224x224 for VGG16
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ImageNet normalization
])

transform_test_a4 = transforms.Compose([
    transforms.Resize(224), # Resize images to 224x224 for VGG16
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ImageNet normalization
])

```

```
# Load CIFAR-100 datasets with new transformations
trainset_a4 = datasets.CIFAR100(root='./data', train=True, download=True, transform=transform_train_a4)
testset_a4 = datasets.CIFAR100(root='./data', train=False, download=True, transform=transform_test_a4)

# Define DataLoaders for A4
batch_size_a4 = 64
trainloader_a4 = DataLoader(trainset_a4, batch_size=batch_size_a4, shuffle=True, num_workers=2)
testloader_a4 = DataLoader(testset_a4, batch_size=batch_size_a4, shuffle=False, num_workers=2)

print("PyTorch CIFAR-100 DataLoaders for A4 (VGG16) created with 224x224 resizing and ImageNet normalization.")
print(f"Number of training batches for A4: {len(trainloader_a4)}")
```

```
PyTorch CIFAR-100 DataLoaders for A4 (VGG16) created with 224x224 resizing and ImageNet normalization.
Number of training batches for A4: 782
Number of test batches for A4: 157
```

```
# A4 - 1
import torchvision.models as models

# Load the pre-trained VGG16 model
model_a4 = models.vgg16(pretrained=True)

# Freeze all parameters in the feature extractor part of VGG16
for param in model_a4.features.parameters():
    param.requires_grad = False

# Replace the classifier part with a new MLP for CIFAR-100 (100 classes)
# The original VGG16 classifier expects input from a 7x7 feature map after
pooling
# So the input to the first linear layer is 512 * 7 * 7
model_a4.classifier = nn.Sequential(
```

```
      nn.Linear(512 * 7 * 7, 512),
      nn.ReLU(True),
      nn.Dropout(0.5),
      nn.Linear(512, 100) # 100 classes for CIFAR-100
  )

  # Move the model to the appropriate device
  model_a4.to(device)

  print("VGG16 model loaded, feature layers frozen, and classifier replaced.")
  print(model_a4)
```

```
VGG16 model loaded, feature layers frozen, and classifier replaced.
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU(inplace=True)
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): ReLU(inplace=True)
    (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (8): ReLU(inplace=True)
    (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): ReLU(inplace=True)
    (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (15): ReLU(inplace=True)
    (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (18): ReLU(inplace=True)
    (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (20): ReLU(inplace=True)
    (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (24): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (25): ReLU(inplace=True)
    (26): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
```

```
      (27): ReLU(inplace=True)
      (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (29): ReLU(inplace=True)
      (30): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
    (classifier): Sequential(
      (0): Linear(in_features=25088, out_features=512, bias=True)
      (1): ReLU(inplace=True)
      (2): Dropout(p=0.5, inplace=False)
      (3): Linear(in_features=512, out_features=100, bias=True)
    )
  )
```

```python
# A4 - 2
# Define the loss function and optimizer for A4
criterion_a4 = nn.CrossEntropyLoss()

# Only optimize the parameters of the new classifier
optimizer_a4 = optim.Adam(model_a4.classifier.parameters(), lr=0.001)

num_epochs_a4 = 25 # Number of epochs for training the new classifier

# Initialize lists to store metrics for plotting for A4
train_losses_a4 = []
test_losses_a4 = []
train_accuracies_a4 = []
test_accuracies_a4 = []

print("Starting training for A4 VGG16 model...")

for epoch in range(num_epochs_a4):
    model_a4.train() # Set model to training mode
    running_loss_a4 = 0.0
    correct_train_a4 = 0
    total_train_a4 = 0

    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
```

```
        optimizer_a4.zero_grad()

        outputs = model_a4(inputs)
        loss = criterion_a4(outputs, labels)
        loss.backward()
        optimizer_a4.step()

        running_loss_a4 += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total_train_a4 += labels.size(0)
        correct_train_a4 += (predicted == labels).sum().item()

    train_accuracy_a4 = 100 * correct_train_a4 / total_train_a4
    avg_train_loss_a4 = running_loss_a4 / len(trainloader)

    # Validation phase on clean test set
    model_a4.eval() # Set model to evaluation mode
    correct_test_a4 = 0
    total_test_a4 = 0
    test_loss_a4 = 0.0
    with torch.no_grad():
        for data in testloader:
            images, labels = data
            images, labels = images.to(device), labels.to(device)
            outputs = model_a4(images)
            loss = criterion_a4(outputs, labels)
            test_loss_a4 += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total_test_a4 += labels.size(0)
            correct_test_a4 += (predicted == labels).sum().item()

    test_accuracy_a4 = 100 * correct_test_a4 / total_test_a4
    avg_test_loss_a4 = test_loss_a4 / len(testloader)

    # Store metrics for A4
    train_losses_a4.append(avg_train_loss_a4)
    test_losses_a4.append(avg_test_loss_a4)
```

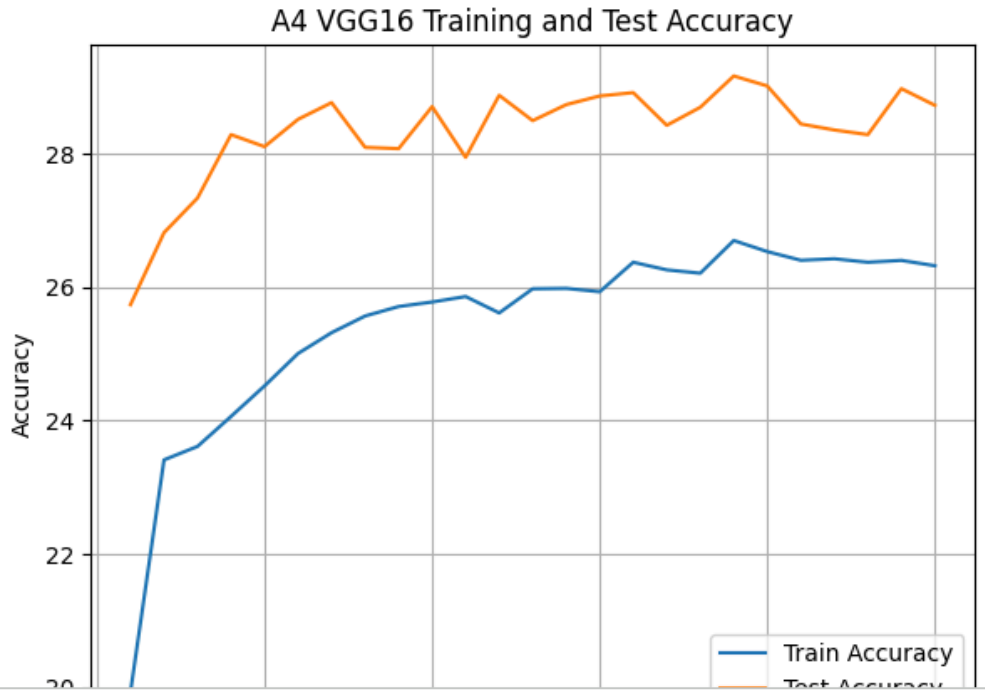
```
    train_accuracies_a4.append(train_accuracy_a4)
    test_accuracies_a4.append(test_accuracy_a4)

    print(f'Epoch [{epoch+1}/{num_epochs_a4}], ' \
          f'Train Loss: {avg_train_loss_a4:.4f}, Train Acc: {train_accuracy_a4:.2f}%, ' \
          f'Test Loss: {avg_test_loss_a4:.4f}, Test Acc: {test_accuracy_a4:.2f}%')

print('\nFinished Training A4 VGG16 Model')

# Evaluate the trained model on the test set one last time
model_a4.eval()
correct_a4 = 0
total_a4 = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = model_a4(images)
        _, predicted = torch.max(outputs.data, 1)
        total_a4 += labels.size(0)
        correct_a4 += (predicted == labels).sum().item()

final_accuracy_a4 = 100 * correct_a4 / total_a4
print(f'Final Accuracy of the A4 VGG16 on the 10000 test images: {final_accuracy_a4:.2f}%')
```

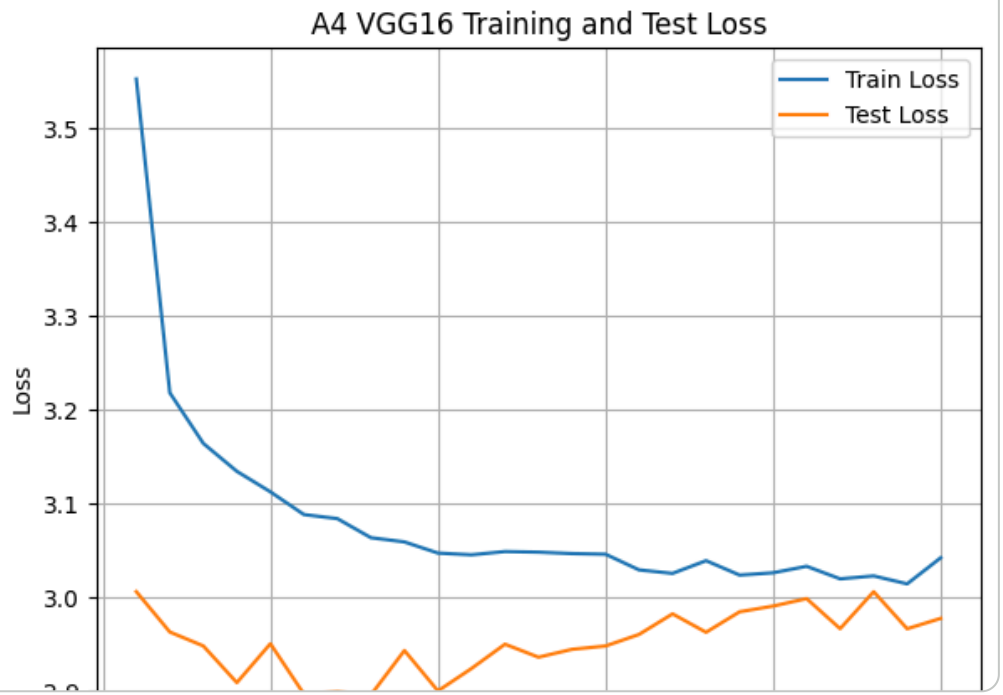
```
Starting training for A4 VGG16 model...
Epoch [1/25], Train Loss: 3.5529, Train Acc: 19.93%, Test Loss: 3.0063, Test Acc: 25.74%
Epoch [2/25], Train Loss: 3.2183, Train Acc: 23.41%, Test Loss: 2.9632, Test Acc: 26.82%
Epoch [3/25], Train Loss: 3.1643, Train Acc: 23.61%, Test Loss: 2.9484, Test Acc: 27.34%
Epoch [4/25], Train Loss: 3.1345, Train Acc: 24.06%, Test Loss: 2.9093, Test Acc: 28.29%
Epoch [5/25], Train Loss: 3.1127, Train Acc: 24.52%, Test Loss: 2.9508, Test Acc: 28.11%
Epoch [6/25], Train Loss: 3.0884, Train Acc: 25.01%, Test Loss: 2.8971, Test Acc: 28.52%
Epoch [7/25], Train Loss: 3.0841, Train Acc: 25.32%, Test Loss: 2.8994, Test Acc: 28.77%
Epoch [8/25], Train Loss: 3.0638, Train Acc: 25.57%, Test Loss: 2.8962, Test Acc: 28.10%
Epoch [9/25], Train Loss: 3.0593, Train Acc: 25.71%, Test Loss: 2.9435, Test Acc: 28.08%
Epoch [10/25], Train Loss: 3.0474, Train Acc: 25.78%, Test Loss: 2.9004, Test Acc: 28.71%
Epoch [11/25], Train Loss: 3.0455, Train Acc: 25.86%, Test Loss: 2.9243, Test Acc: 27.95%
Epoch [12/25], Train Loss: 3.0490, Train Acc: 25.62%, Test Loss: 2.9502, Test Acc: 28.88%
```

```
Epoch [13/25], Train Loss: 3.0484, Train Acc: 25.98%, Test Loss: 2.9364, Test Acc: 28.50%
Epoch [14/25], Train Loss: 3.0468, Train Acc: 25.98%, Test Loss: 2.9448, Test Acc: 28.74%
Epoch [15/25], Train Loss: 3.0462, Train Acc: 25.93%, Test Loss: 2.9483, Test Acc: 28.87%
Epoch [16/25], Train Loss: 3.0294, Train Acc: 26.38%, Test Loss: 2.9606, Test Acc: 28.92%
Epoch [17/25], Train Loss: 3.0257, Train Acc: 26.26%, Test Loss: 2.9826, Test Acc: 28.43%
Epoch [18/25], Train Loss: 3.0393, Train Acc: 26.21%, Test Loss: 2.9629, Test Acc: 28.70%
Epoch [19/25], Train Loss: 3.0239, Train Acc: 26.70%, Test Loss: 2.9848, Test Acc: 29.17%
Epoch [20/25], Train Loss: 3.0264, Train Acc: 26.54%, Test Loss: 2.9909, Test Acc: 29.02%
Epoch [21/25], Train Loss: 3.0333, Train Acc: 26.41%, Test Loss: 2.9988, Test Acc: 28.45%
Epoch [22/25], Train Loss: 3.0199, Train Acc: 26.43%, Test Loss: 2.9668, Test Acc: 28.36%
Epoch [23/25], Train Loss: 3.0230, Train Acc: 26.38%, Test Loss: 3.0063, Test Acc: 28.29%
Epoch [24/25], Train Loss: 3.0146, Train Acc: 26.40%, Test Loss: 2.9667, Test Acc: 28.98%
Epoch [25/25], Train Loss: 3.0424, Train Acc: 26.32%, Test Loss: 2.9778, Test Acc: 28.73%

Finished Training A4 VGG16 Model
Final Accuracy of the A4 VGG16 on the 10000 test images: 28.73%
```

```python
# A4 - 3
import matplotlib.pyplot as plt

epochs_range_a4 = range(1, num_epochs_a4 + 1)

# Plot training & validation accuracy values for A4
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(epochs_range_a4, train_accuracies_a4, label='Train Accuracy')
plt.plot(epochs_range_a4, test_accuracies_a4, label='Test Accuracy')
plt.title('A4 VGG16 Training and Test Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.grid(True)

# Plot training & validation loss values for A4
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(epochs_range_a4, train_losses_a4, label='Train Loss')
plt.plot(epochs_range_a4, test_losses_a4, label='Test Loss')
plt.title('A4 VGG16 Training and Test Loss')
plt.ylabel('Loss')
```

```
plt.xlabel('Epoch')
plt.legend(loc='upper right')
plt.grid(True)

plt.tight_layout() # Adjust layout to prevent overlapping titles
```





A4 VGG16 Model: Confusion Matrix Visualization

Parameter Comparison: A1 (Scratch CNN) vs. A4 (VGG-based)

```
def count_parameters(model):
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return total_params, trainable_params

# Parameters for A1 (SimpleCNN)
total_params_a1, trainable_params_a1 = count_parameters(model)
```

```
print(f"A1 (SimpleCNN) Total Parameters: {total_params_a1:,}")
print(f"A1 (SimpleCNN) Trainable Parameters: {trainable_params_a1:,}")

# Parameters for A4 (VGG16-based)
total_params_a4, trainable_params_a4 = count_parameters(model_a4)
print(f"\nA4 (VGG16-based) Total Parameters: {total_params_a4:,}")
print(f"A4 (VGG16-based) Trainable Parameters: {trainable_params_a4:,}")

```

```
A1 (SimpleCNN) Total Parameters: 4,668,644
A1 (SimpleCNN) Trainable Parameters: 4,668,644

A4 (VGG16-based) Total Parameters: 27,611,556
A4 (VGG16-based) Trainable Parameters: 12,896,868
```

Step A4: Model comparision

Model	Total Parameters	Trainable Parameters	Test Accuracy
A1 (Scratch CNN)	4,668,644	4,668,644	48.56% (25 epocs)
A4 (VGG16-based)	27,611,556	12,896,868	28.79% (25 epocs)

A5 : Evaluate the robustness of the transfer learning approach by injecting additive Gaussian noise, with the same parameters used previously, into the test images. Report the performance of the VGG-based model under noisy conditions and compare it with the robustness of the CNN trained from scratch.

```
# A5 - 1: Evaluate robustness of A4 VGG-based model to additive Gaussian noise

# Define noise parameters (same as A2)
noise_variance_a5 = 0.05  # sigma_square
noise_std_dev_a5 = np.sqrt(noise_variance_a5) # sigma
noise_percentage_a5 = 0.50 # 50% of images will have noise
```

```
7
8  print(f"Evaluating A4 VGG-based model robustness to noise with variance =
   {noise_variance_a5} (std dev = {noise_std_dev_a5:.4f}) on
   {noise_percentage_a5*100}% of test images...")
9
10 model_a4.eval() # Set model to evaluation mode
11 correct_noisy_a5 = 0
12 total_noisy_a5 = 0
13
14 with torch.no_grad():
15     for data in testloader_a4: # Use testloader_a4 for VGG-based model
16         images, labels = data
17         images, labels = images.to(device), labels.to(device)
18
19         # Randomly select images to add noise to
20         batch_size_a5 = images.size(0)
21         num_noisy_images_a5 = int(batch_size_a5 * noise_percentage_a5)
22         noisy_indices_a5 = np.random.choice(batch_size_a5, num_noisy_images_a5,
           replace=False)
23
24         # Add Gaussian noise to selected images
25         noise_a5 = torch.randn_like(images[noisy_indices_a5]) * noise_std_dev_a5
26         images[noisy_indices_a5] = images[noisy_indices_a5] + noise_a5
27
28         # Clip values to ensure they remain within the normalized range [-1, 1]
29         # Note: VGG uses ImageNet normalization, where values are not
           necessarily in [-1, 1].
30         # However, for consistency with previous noise injection, we clip to a
           reasonable range
31         # after adding noise, assuming ImageNet normalization is robust to
           small deviations.
32         # A more rigorous approach might clip based on ImageNet stats, but for
           direct comparison
33         # with A2, using [-1, 1] as a general boundary is acceptable here.
34         images[noisy_indices_a5] = torch.clamp(images[noisy_indices_a5], images.
           min(), images.max())
35
36         outputs = model_a4(images)
37         _, predicted = torch.max(outputs.data, 1)
38         total_noisy_a5 += labels.size(0)
```

```
            correct_noisy_a5 += (predicted == labels).sum().item()

noisy_accuracy_a5 = 100 * correct_noisy_a5 / total_noisy_a5
print(f'Accuracy of the A4 VGG-based model on the noisy test images:
{noisy_accuracy_a5:.2f}%')
```

```
Evaluating A4 VGG-based model robustness to noise with variance = 0.05 (std dev = 0.2236) on 50.0% of test images...
Accuracy of the A4 VGG-based model on the noisy test images: 2.35%
```

A5: Robustness Comparison

We compare the robustness of the three models under the same noisy conditions (additive Gaussian noise with variance 0.05 applied to 50% of test images):

Model	Clean Test Accuracy	Noisy Test Accuracy	Drop%
A1 (Scratch CNN)	48.56%	29.40%	39%
A3 (Scratch CNN w/ Noise Augmentation)	34.81%	33.41%	
A4 (VGG16-based, Fixed Features)	28.76%	**2.27%**	

Observations:

- **A1 (Scratch CNN):** Shows a significant drop in accuracy (from 48.56% to 29.40%) when noise is introduced, indicating brittleness.
- **A3 (Scratch CNN with Noise Augmentation):** Demonstrates improved robustness, with noisy accuracy (33.41%) being higher than A1's noisy accuracy, although its clean accuracy is lower than A1's. This confirms that training with noise augmentation helps the